



CORE COMPUTER SCIENCE

Object-Oriented Programming

Placement Preparation Notes

Created by Pranav Gawai

Object-Oriented Programming Roadmap

PLACEMENT PREPARATION GUIDE

How to use this Handbook:

Object-Oriented Programming is heavily tested on polymorphism binding, access modifier scopes, SOLID principles, and lifecycle patterns (constructors/destructors). Focus on memory structures (heap vs stack, VTABLE lookups) and C++ vs Java design decisions.

Three-Phase Learning Plan

PHASE 1

Classes & Pillars

Master class/object memory model, constructors, and the 4 pillars (inheritance, polymorphism, abstraction, encapsulation). (Topics 1 – 6)

PHASE 2

Method Binding & Design

Study method operations, VTABLE dynamic dispatch, abstract interfaces, aggregation links, and SOLID guidelines. (Topics 7 – 12)

PHASE 3

Memory & Language details

Learn access scopes, static binding resolution, garbage collection algorithms, copy patterns, and C++ copy constructors. (Topics 13 – 17)

Target Completion: 1.5 Hours Core Study & 30 Mins Self-Recall

Created by **Pranav Gawai**
grindos.pranavx.in

Table of Contents

Object-Oriented Programming · Placement Preparation Handbook

WHAT IS OOP?

Object-Oriented Programming (OOP) is a programming paradigm structured around **Objects** rather than functions or logic. It models real-world entities to produce modular, maintainable, and scalable software.

THE 4 CORE PILLARS

1. **Encapsulation:** Hiding internal state via private variables and exposing controlled accessors.
2. **Inheritance:** Establishing IS-A relations to reuse behaviors and properties.
3. **Polymorphism:** Responding to messages differently at compile-time (overload) or runtime (override).
4. **Abstraction:** Defining high-level interfaces to decouple contract from execution.

PLACEMENT FOCUS

Top product-based interviewers heavily grill candidates on **runtime override mechanics (VTABLE/VPTR)**, constructor/destructor execution chains, **SOLID principles**, and avoiding diamond inheritance traps.

OOP FUNDAMENTALS

- 01 Class & Object p.04
- 02 Constructor & Destructor p.05

THE FOUR PILLARS

- 03 Pillar 1: Inheritance p.06
- 04 Pillar 2: Polymorphism p.07
- 05 Pillar 3: Abstraction p.08
- 06 Pillar 4: Encapsulation p.09

METHOD OPERATIONS

- 07 Overloading vs Overriding p.10
- 08 Virtual Function & VTABLE p.11
- 14 Static vs Dynamic Binding p.17

ABSTRACT CONTRACTS

- 09 Abstract Class vs Interface p.12

OBJECT RELATIONSHIPS

- 10 Aggregation & Composition p.13

SOLID PRINCIPLES

- 11 SOLID: S & O Principles p.14
- 12 SOLID: L, I, & D Principles p.15

LANGUAGE SPECIFICS

- 13 Access Modifiers p.16
- 17 Copy Const. & Assignment p.20

MEMORY & MANAGEMENT

- 15 Garbage Collection & Memory p.18
- 16 Shallow vs Deep Copy p.19

OOP FUNDAMENTALS

Yield: ★★★★★

01 – Class & Object

The foundational blueprint and instance paradigms of OOP.

CORE DEFINITION

Class: A user-defined data type acting as a blueprint. Occupies no memory at runtime. Stores method templates and member definitions.

Object: A basic runtime entity. An instance of a class. Occupies physical memory space on the heap or stack.

Class blueprint definition:

```
class Car { String model; int year; }
```

Object instantiation (RAM):

```
Car myCar = new Car("Tesla", 2024);  
(Allocates memory for 'model' and 'year')
```

CORE INTERVIEW QUESTION

"Does a Class occupy memory space? How does it differ from an Object in memory?"

KEYWORDS TO DROP

LOGICAL BLUEPRINT

PHYSICAL ENTITY

INSTANCE CREATION

HEAP ALLOCATION

MODEL ANSWER

"No, a Class is a logical template that does not occupy physical RAM. Memory is allocated only when an Object (an instance of the class) is created. The object's member variables are loaded into the heap or stack at execution time."

DEEP DIVE FOLLOW-UP

→ "What is an anonymous object?"

An object created without a reference variable (e.g. `new Car().drive();`). It is immediate garbage collected after the statement executes.

MEMORY ALLOCATION DEPTH

Level 1 (Logical): Class acts as variable template blueprint.

Level 2 (Stack): Local references held in thread execution stack.

Level 3 (Heap): Instance objects created in dynamic heap space.

Level 4 (Method): Static variables and code templates loaded to Metaspace.

WHERE USED IN INDUSTRY

Enterprise frameworks (like Spring or Hibernate) use class blueprints to dynamically instantiate beans and map database rows to Java objects via reflection.

INTERVIEW DEPTH

- L1: Class vs Object blueprint definition.
- L2: Differences in memory allocation (Stack vs Heap).
- L3: Class loader subsystem and Metaspace loading.
- L4: Overhead of object headers (Mark Word, Klass Word).
- L5: Object pooling pattern to avoid garbage collection.

COMMON MISTAKE

Saying classes occupy memory. Only objects occupy physical RAM (except static class definitions).

TOP FOLLOW-UPS

- What is the difference between class and struct?
- What is an anonymous object?

INTERVIEWER TRAP

Don't say static variables belong to the object. Static variables are shared at the class level and exist in global memory, not inside the object's instance space.

MEMORY TRICK

Class = Architectural blueprint. Object = The actual physical house built from the blueprint.

OOP FUNDAMENTALS

Yield: ★★★★★

02 – Constructor & Destructor

Initializing state during allocation and reclaiming resources during deallocation.

LIFECYCLE METHODS

Constructor: Special member function called automatically when an object is created. Has same name as class, no return type. Can be default, parameterized, or copy constructor.

Destructor: Called automatically when an object goes out of scope or is deleted. Cleans up memory, files, or database locks. Preceded by tilde (~) in C++.

Constructor Chaining Execution Order:

1. Parent static blocks
2. Child static blocks
3. Parent instance initializers & Constructor
4. Child instance initializers & Constructor

CONSTRUCTOR DELEGATION (C++11/JAVA)

Allows a constructor to call another constructor in the same class (using `this()` in Java or initializer lists in C++) to prevent duplicate initialization code.

WHERE USED IN INDUSTRY

Thread pools (like `ExecutorService`) and database connection pools (like `HikariCP`) use constructors to pre-allocate connections and cleanup methods to safely release sockets.

INTERVIEW DEPTH

- L1: Purpose of constructor and destructor lifecycle.
- L2: Initialization block execution vs constructor code.
- L3: Constructor chaining, virtual base construction in C++.
- L4: Exceptions in constructors and resource safety (RAII).
- L5: Placement of destructors in smart pointers and standard containers.

CORE INTERVIEW QUESTION

"Can a Constructor be private? What happens if you define a private constructor?"

KEYWORDS TO DROP

SINGLETON DESIGN

INSTANCE PREVENTION

FACTORY METHOD

STATIC BUILDER

MODEL ANSWER

"Yes, a constructor can be private. This prevents external code from instantiating the class using `new`. It is commonly used in the **Singleton Design Pattern** or utility classes, where access to the single instance is controlled via a static factory method."

DEEP DIVE FOLLOW-UP

→ "Does Java support destructors?"

No. Java manages memory automatically using a garbage collector. C++ requires explicit destructors to free heap allocations and prevent memory leaks.

REAL INTERVIEW SCENARIO

Interviewer: "What happens if a constructor throws an exception?"

Candidate: "The object is partially constructed. The destructor will not run automatically, which can cause memory leaks unless resources are wrapped in smart pointers or cleaned up explicitly."

⚠ COMMON MISTAKE

Thinking Java/Python has destructors similar to C++. They rely on automated garbage collectors.

🔄 TOP FOLLOW-UPS

- What is constructor delegation?
- Can constructors be overloaded?

🔴 INTERVIEWER TRAP

Don't forget that if you define a parameterized constructor, the compiler stops generating the default no-argument constructor automatically.

💡 MEMORY TRICK

Constructor = Birth certificate setup. Destructor = Will execution and property cleanup.

THE FOUR PILLARS

Yield: ★★★★★

O3 – Pillar 1: Inheritance

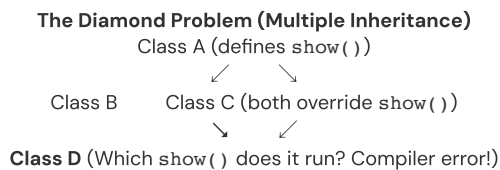
Creating hierarchical relationships to reuse class code structures.

CODE REUSE

Allows a child class (subclass) to inherit fields and methods from a parent class (superclass), establishing an ****IS-A**** relationship.

Types of Inheritance:

Single, Multiple (interfaces in Java), Multi-level, Hierarchical, Hybrid.



CORE INTERVIEW QUESTION

"Why does Java block multiple inheritance of classes, and how does it resolve the Diamond Problem?"

KEYWORDS TO DROP

DIAMOND PROBLEM

AMBIGUITY RESOLUTION

INTERFACE DEFAULT

METHOD SIGNATURE

MODEL ANSWER

"Java blocks multiple inheritance of classes to prevent compiler ambiguity (the Diamond Problem). If two parents define the same method, the child doesn't know which to execute. Java resolves this by supporting multiple inheritance only through ****Interfaces****, where the child must explicitly override conflicting default methods."

DEEP DIVE FOLLOW-UP

→ **"What is the difference between IS-A and HAS-A?"**

IS-A represents inheritance (e.g. Dog is an Animal).
HAS-A represents composition/association (e.g. Dog has a Collar reference).

INHERITANCE DEPTH

Level 1 (Basic): Single and multilevel class inheritance hierarchy.

Level 2 (Conflict): Resolve the Diamond Problem using virtual keywords.

Level 3 (Contract): Leverage interfaces to implement multiple inheritances.

Level 4 (Dynamic): JVM method table resolution and super delegation.

UPCASTING VS DOWNCASTING SAFETY

Upcasting: Casting child to parent (e.g. `Animal a = new Dog();`). Safe and implicit.

Downcasting: Casting parent to child (e.g. `Dog d = (Dog) a;`). Unsafe; throws `ClassCastException` unless verified using `instanceof`.

WHERE USED IN INDUSTRY

UI frameworks (like Android SDK or iOS UIKit) structure views in deep hierarchy trees (e.g. `TextView` inherits from `View`) to inherit layout characteristics.

INTERVIEW DEPTH

L1: Code reuse using subclassing/super delegation.

L2: Overriding vs inheriting fields (no polymorphism for fields).

L3: Virtual base class pointer offsets in C++ memory layout.

L4: Tight coupling vulnerability (Fragile Base Class problem).

L5: Composition-over-inheritance design pattern implementations.

COMMON MISTAKE

Believing Java supports multiple class inheritance. It only supports multiple interface implementations.

INTERVIEWER TRAP

Don't state that interfaces in Java have no implementation code anymore. Java 8 introduced `default` and `static` methods that can have active code blocks.

TOP FOLLOW-UPS

- What is virtual base inheritance?
- Why does Java avoid multiple class inheritance?

MEMORY TRICK

Inheritance is like inheriting your parents' house (ready to use), while interfaces are architectural guidelines you must construct yourself.

THE FOUR PILLARS

Yield: ★★★★★

04 – Pillar 2: Polymorphism

Processing objects differently based on their data type or class.

BEHAVIORAL ADAPTION

Polymorphism means 'many forms'. It allows a single interface or method signature to act differently depending on the context.

Type	Binding Stage	Mechanism
Compile-Time	Static (Compile)	Method Overloading
Run-Time	Dynamic (Execution)	Method Overriding

DYNAMIC METHOD DISPATCH MECHANICS

When an overridden method is called, the runtime checks the actual object type in memory. The CPU resolves the call using a method reference array (VTABLE). This dereference step is why overriding adds a tiny performance penalty compared to static overloading.

WHERE USED IN INDUSTRY

JDBC drivers implement interfaces (like Connection) polymorphically; the same connection code behaves differently for MySQL, PostgreSQL, or Oracle at runtime.

INTERVIEW DEPTH

- L1:** Definition of overloading (static) vs overriding (dynamic).
- L2:** Binding stage differences: compile-time vs runtime execution.
- L3:** VTABLE virtual method resolution and method dispatch mechanism.
- L4:** Performance costs of CPU cache misses during VTABLE lookups.
- L5:** Polymorphic interface adapters in decoupled enterprise architectures.

CORE INTERVIEW QUESTION

"Explain Run-Time Polymorphism. How does the JVM determine which method to execute at runtime?"

KEYWORDS TO DROP

UPCASTING

DYNAMIC METHOD DISP

OVERRIDE LOOKUP

VTABLE REFERENCE

MODEL ANSWER

"Run-time polymorphism (Dynamic Method Dispatch) occurs when a parent reference points to a child object. At compile time, the compiler checks the parent reference type. At runtime, the JVM looks up the actual object's type on the heap and executes its overridden method."

DEEP DIVE FOLLOW-UP

→ "Can static methods be overridden?"

No. Static methods belong to the class, not instance. Attempting to override a static method results in **Method Hiding****, resolved at compile time.**

POLYMORPHISM BIND DEPTH

Level 1 (Define): Understand overloading (static) vs overriding (dynamic).

Level 2 (Static): Overloading resolved at compile time via argument signature matches.

Level 3 (Dynamic): Overriding resolved at runtime by inspecting heap object types.

Level 4 (Cost): Dereferencing VTABLE pointer adds CPU branch target cache penalties.

COMMON MISTAKE

Saying static polymorphism uses virtual tables. VTABLEs are exclusive to dynamic runtime overriding dispatch.

TOP FOLLOW-UPS

- Differentiate compile-time vs run-time polymorphism.
- What is method hiding?

INTERVIEWER TRAP

Don't confuse overloading with overriding. Overloading uses different signatures in the same class. Overriding uses identical signatures in child classes.

MEMORY TRICK

Overloading = same tool, different attachments. Overriding = same button, different actions on child devices.

THE FOUR PILLARS

Yield: ★★★★★

05 – Pillar 3: Abstraction

Hiding implementation details, showing only essential properties.

COMPLEXITY CONTROL

Abstraction focuses on **what** an object does rather than **how** it does it, reducing design complexity. Implemented using Abstract Classes and Interfaces.

Abstraction Levels:

Concrete Class (0%) → Abstract Class (0-100%) → Interface (100% Pure Abstraction)

INTERFACE CONTRACTS IN ARCHITECTURE

Interfaces decouple modules. By programming to an interface (e.g. `List` rather than `ArrayList`), you can swap implementations without breaking application modules.

WHERE USED IN INDUSTRY

Cloud SDKs (like AWS S3 SDK) expose abstract interfaces (`S3Client`) to decouple client business logic from raw network call implementations.

INTERVIEW DEPTH

- L1: Hiding background details using high-level declarations.
- L2: Decoupled abstract interface vs detailed implementation class.
- L3: Compilation firewall (Pimpl idiom in C++) to prevent rebuilds.
- L4: Over-engineering risk and API evolution backward-compatibility.
- L5: Exposing stable service interfaces to client microservices.

CORE INTERVIEW QUESTION

"How does Abstraction differ from Encapsulation? Aren't they both hiding data?"

KEYWORDS TO DROP

DESIGN LEVEL HIDE

IMPLEMENTATION LEVEL

DATA WRAPPING

COMPLEXITY REDUCTION

MODEL ANSWER

"Abstraction hides implementation details at the **design level** to focus on interface behaviors. Encapsulation hides data variables at the **implementation level** by wrapping variables and methods into a single class, protecting state from unauthorized modification."

DEEP DIVE FOLLOW-UP

→ **"What is a concrete class?"**

A standard class that implements all its methods and can be directly instantiated (unlike abstract classes or interfaces).

REAL INTERVIEW SCENARIO

Interviewer: "How does Abstraction help in scale systems?"

Candidate: "By defining clean interface boundaries. If we code to interfaces, we can swap database engines or payment gateways without changing core business logic modules."

COMMON MISTAKE

Confusing Abstraction (design-level design hiding) with Encapsulation (data-level hiding).

TOP FOLLOW-UPS

- Can an abstract class implement an interface?
- What is a concrete class?

INTERVIEWER TRAP

Don't say abstract classes are identical to interfaces. Abstract classes can hold private state variables and concrete constructors; interfaces are strictly design contracts.

MEMORY TRICK

Abstraction = The dashboard interface. Encapsulation = The locked hood protecting the engine parts.

THE FOUR PILLARS

Yield: ★★★★★

O6 – Pillar 4: Encapsulation

Wrapping data variables and methods into a single protective unit.

DATA PROTECTION

Prevents direct access to data fields from outside the class boundary. Member variables are marked ****private****, and accessed only through ****public getter/setter methods****.

Encapsulated Class Wrapper

```
class BankAccount {  
  
    private  
    double balance;  
  
    public  
    void deposit(double val) {  
        if (val > 0) balance += val; // validation check  
    }  
}
```

THREAD SAFETY VIA ENCAPSULATION

By hiding fields behind private modifiers, we prevent concurrent threads from modifying class state directly. We can add synchronization locks inside getter/setter methods to enforce thread-safety rules.

WHERE USED IN INDUSTRY

REST API request/response payloads (DTOs) encapsulate raw JSON fields using private properties and validate input formats inside getter/setter methods.

INTERVIEW DEPTH

- L1:** Protecting state using private fields and public get/set accessors.
- L2:** Data hiding vs encapsulation (hiding implementation complexity).
- L3:** Reference escape prevention by returning defensive deep copies.
- L4:** Performance overhead of getter/setter boilerplate code.
- L5:** Enforcing domain invariants inside Domain-Driven Design aggregates.

CORE INTERVIEW QUESTION

"What is the benefit of using Getter/Setter methods instead of making class variables public?"

KEYWORDS TO DROP

DATA VALIDATION

READONLY PROPERTY

ACCESS RESTRICTION

STATE GUARDING

MODEL ANSWER

"Public getters/setters let us control data entry through validation rules, preventing invalid states (e.g. setting age to -5). They also allow making variables read-only by omitting setters, protecting the class's internal state."

DEEP DIVE FOLLOW-UP

→ **"Is encapsulation broken if a getter returns a mutable object reference?"**

Yes. Returning direct references to mutable objects allows external code to modify class state. Getters should return deep copies instead.

STATE PROTECTION DEPTH

- Level 1 (Scope):** Declare variables private, providing getters.
- Level 2 (Invariants):** Validate parameter inputs inside setters.
- Level 3 (Read-Only):** Omit setters to implement immutable states.
- Level 4 (Escape):** Prevent reference escape by returning deep clones.

COMMON MISTAKE

Believing encapsulation is just wrapping variables. It is about data hiding to prevent unauthorized modifications.

TOP FOLLOW-UPS

- Is encapsulation broken if getters return mutable references?
- Explain access control levels.

INTERVIEWER TRAP

Don't confuse encapsulation with security encryption. Encapsulation is a design pattern for structure control, not cryptographic protection.

MEMORY TRICK

Encapsulation is like a medical capsule: it wraps the ingredients to protect them from external elements.

METHOD OPERATIONS

Yield: ★★★★★

07 – Overloading vs Overriding

Different method signatures in one class vs matching signatures in subclasses.

BEHAVIOR MAPPING

Overloading: Multiple methods in the same class share a name but have different parameter counts or types. Return type changes alone are not allowed.

Overriding: A subclass redefines a method from its parent class using the exact same signature (name, parameter list, and return type).

Property	Overloading	Overriding
Scope	Same class	Child subclasses
Arguments	Must change	Must match exactly
Return Type	Can differ	Must be covariant
Access Level	Any	Must not restrict

COVARIANT RETURN TYPES

During overriding, a child subclass method is allowed to return a subtype of the return type declared in the parent class (e.g., parent returns `Shape`, child returns `Circle`).

WHERE USED IN INDUSTRY

Logging libraries (like Log4j) overload method signatures to accept varying arguments (`String`, `Exception`, `Object[]`) for runtime logging convenience.

INTERVIEW DEPTH

- L1: Overloading (signature change) vs Overriding (behavior change).
- L2: Return type rules: covariant returns in overriding methods.
- L3: JVM resolution of static vs virtual method dispatch opcodes.
- L4: Fragile subclassing if base class overloads are overridden.
- L5: API design convenience vs maintaining code base readability.

CORE INTERVIEW QUESTION

"Why can't you overload a method in Java by changing **ONLY** the return type?"

KEYWORDS TO DROP

SIGNATURE AMBIGUITY

CALL RESOLUTION

RETURN TYPE EXCL.

COMPILER ERROR

MODEL ANSWER

"Because call resolution is based on parameters, not return values. If you call a method `calculate()` without assigning the result, the compiler cannot resolve which version to run based on return type alone, causing a compile error."

DEEP DIVE FOLLOW-UP

→ "What is a covariant return type in overriding?"

An overriding method in a subclass is allowed to return a subtype of the class returned by the parent method (e.g. parent returns `Shape`, child returns `Circle`).

OPERATOR OVERLOADING CONTRAST

C++ Support: Allows overloading standard operators (like `+` or `<<`) to enhance custom class utility.

Java Exclusion: Excludes user-level operator overloading (except `+` for `String`) to prevent compiler complexity and maintain code readability.

COMMON MISTAKE

Changing only return types to overload a method. Overloading requires unique argument signatures; return type changes alone fail compile checks.

TOP FOLLOW-UPS

- Explain covariant return types.
- Can static methods be overridden?

INTERVIEWER TRAP

Don't forget: Overriding methods cannot throw broader checked exceptions than the parent method, nor can they reduce access permissions (e.g. `public` cannot become `private`).

MEMORY TRICK

Overload = Same method name, different input parameters.
Override = Child rewriting parent rules.

METHOD OPERATIONS

Yield: ★★★★★

O8 – Virtual Function & VTABLE

Analyzing C++ dynamic dispatch using virtual method tables.

DYNAMIC DISPATCH

C++ uses a **Virtual Table (VTABLE)**** to resolve function addresses at runtime during dynamic dispatch. If a class defines a virtual function: the compiler inserts a hidden pointer (`_vptr`) pointing to the class's static VTABLE containing method pointers.

VTABLE Lookup Flow

Object Instance (contains `_vptr`)
↓ points to
Class VTABLE [MethodA *ptr, MethodB *ptr]
↓ executes
Heap memory target function address

VIRTUAL DESTRUCTORS IMPORTANCE

If a parent class base pointer deletes a child class instance, and the parent destructor is not declared `virtual`, only the parent destructor runs. The child's destructor is bypassed, causing memory leaks.

WHERE USED IN INDUSTRY

Operating system kernels (written in C++) and high-performance game engines (like Unreal Engine) optimize virtual tables to dispatch event handlers with minimal CPU overhead.

INTERVIEW DEPTH

- L1:** Dynamic binding using a hidden pointer in the object.
- L2:** Direct method calls vs indirect pointer dereferencing calls.
- L3:** Object offset 0 virtual pointer (`_vptr`) mapping to class VTABLE.
- L4:** Memory overhead of storing virtual pointer inside small objects.
- L5:** High-speed assembly jumps and compiler devirtualization tricks.

CORE INTERVIEW QUESTION

"Explain how a VTABLE resolves dynamic dispatch at runtime for a child class overriding parent methods."

KEYWORDS TO DROP

VPTR OFFSET

STATIC ARRAY PTR

DYNAMIC BINDING

OBJECT OVERHEAD

MODEL ANSWER

"When a class defines a virtual function, the compiler inserts a hidden `_vptr` into the object. At runtime, when calling `base->draw()`, the CPU reads `_vptr` to find the class's VTABLE, reads the method pointer at the correct offset, and jumps to that address, executing the child's override."

DEEP DIVE FOLLOW-UP

→ **"Why do we need virtual destructors?"**

If a base pointer deleting a child class lacks a virtual destructor, only the base destructor runs, failing to free child-specific allocations and causing memory leaks.

DISPATCH MECHANICS DEPTH

- Level 1 (Object):** Object allocation inserts a hidden `_vptr` at offset 0.
- Level 2 (Table):** Compiler builds a static VTABLE of pointers for the class.
- Level 3 (Lookup):** Target call queries `_vptr` to locate the class's VTABLE.
- Level 4 (Jump):** CPU reads pointer from VTABLE index and performs a jump.

COMMON MISTAKE

Thinking every object has its own unique VTABLE copy. VTABLE is class-static; objects only store a pointer to it (`_vptr`).

TOP FOLLOW-UPS

- What is the overhead of a virtual pointer?
- Draw VTABLE lookup mechanism.

INTERVIEWER TRAP

Don't say virtual functions have zero overhead. Reading `_vptr` and dereferencing the VTABLE adds a small delay and increases object size by 8 bytes (for the pointer).

MEMORY TRICK

VTABLE is a directory of function pointers matching class methods to physical RAM code addresses.

ABSTRACT CONTRACTS

Yield: ★★★★★

09 – Abstract Class vs Interface

Choosing partial implementations vs strict behavioral contracts.

STRUCTURAL CONTRACTS

Abstract Class: Represents a partial template. Can hold state variables, private fields, and constructors. Supports single inheritance only.

Interface: Represents a strict behavioral contract. Variables are static constants. Supports multiple implementation links.

Feature	Abstract Class	Interface
Multiple Inher.	No (Single Base)	Yes (Multiple Impl)
Variables	Any access modifier	Must be static final
Constructors	Yes	No
Speed	Fast (Direct call)	Slower (Interface lookup)

MARKER INTERFACES (SIGNAL FLAGS)

Marker interfaces have zero methods (e.g. `Cloneable`, `Serializable`). They act as a type flag, informing the runtime engine that the class supports specific behaviors or VM optimizations.

WHERE USED IN INDUSTRY

Spring Security uses interfaces (`UserDetailsService`) for contract definition and abstract classes (`WebSecurityConfigurerAdapter`) to provide default configurations.

INTERVIEW DEPTH

- L1: Abstract classes (some state) vs interfaces (pure contracts).
- L2: Single class inheritance vs multiple interface implementation limits.
- L3: Abstract class fields on heap vs interface constants in constant pool.
- L4: Interface pollution vs abstract class code duplication.
- L5: Design pattern selection (e.g. Strategy vs Template method).

CORE INTERVIEW QUESTION

"When would you choose an Abstract Class over an Interface in application architecture?"

KEYWORDS TO DROP

CODE SHARING

STATE VARIABLES

INTERFACE SEGREG.

TIGHT COUPLING

MODEL ANSWER

"Choose an abstract class when subclasses share common implementation code and state variables (e.g. `BaseDatabaseConnector`). Choose an interface to define a decoupled behavior contract implemented by unrelated classes (e.g. `Serializable`)."

DEEP DIVE FOLLOW-UP

→ "What is a marker interface?"

An empty interface with no methods (e.g. `Cloneable` in Java) used to signal metadata flags to the runtime engine.

REAL INTERVIEW SCENARIO

Interviewer: "Why did Java 8 introduce default interface methods?"

Candidate: "To allow adding new methods to existing interfaces without breaking all their implementing classes, enabling backward-compatible library extension."

COMMON MISTAKE

Thinking modern interfaces can't have method bodies. Java 8+ interfaces allow default/static methods, but they still cannot hold state.

TOP FOLLOW-UPS

- What is a marker interface?
- Why did Java 8 introduce default methods?

INTERVIEWER TRAP

Don't say interfaces cannot have any code in Java anymore; since Java 8, they support `default` and `static` methods.

MEMORY TRICK

Abstract class is parent-child relationship (hereditary). Interface is a certification badge (anyone can apply and pass).

OBJECT RELATIONSHIPS

Yield: ★★★★★

10 – Aggregation & Composition

Distinguishing weak references from parent-owned lifecycles.

RELATIONSHIP LEVELS

Association: General link between objects (e.g. Teacher and Student). Can be one-to-one or one-to-many.

Aggregation: Weak HAS-A link. Objects exist independently; parent deletion does not destroy the child (e.g. Department and Professor).

Composition: Strong HAS-A link. Parent owns child lifecycle; deleting the parent automatically destroys the child (e.g. House and Room).

UML Associations Symbols:

- **Aggregation:** Parent <— Child (Empty Diamond)
- **Composition:** Parent ◆— Child (Filled Diamond)

CORE INTERVIEW QUESTION

"How does Composition differ from Aggregation in code structure and lifecycle dependency?"

KEYWORDS TO DROP

LIFECYCLE OWNERSHIP

WEAK ASSOCIATION

OBJECT DESTRUCTION

NESTED INSTANTIATION

MODEL ANSWER

"In Aggregation, the child object is created outside the parent and passed in, existing independently. In Composition, the child is created inside the parent, meaning its lifecycle is tied to the parent's: deleting the parent destroys the child."

DEEP DIVE FOLLOW-UP

→ "Why is Composition preferred over Inheritance?"
Composition keeps classes loosely coupled, allowing behaviors to be changed dynamically at runtime by swapping nested references, avoiding rigid inheritance trees.

RELATIONSHIP DEPTH

Level 1 (Association): Bidirectional or unidirectional links (e.g. Student–Course).

Level 2 (Aggregation): Weak link; objects exist independently (e.g. Employee–Office).

Level 3 (Composition): Strong parent ownership; nested instances (e.g. Document–Page).

Level 4 (Coupling): Composition preferred over subclassing to decouple systems.

FAVORS COMPOSITION OVER INHERITANCE

Inheritance creates tight compile-time coupling (white-box reuse). Composition creates loose coupling by combining objects at runtime (black-box reuse), allowing dynamic swapping of nested references.

WHERE USED IN INDUSTRY

Domain-driven design (DDD) maps database relations using associations: composition for nested entities (Order–LineItem), aggregation for lookup items (Product–Category).

INTERVIEW DEPTH

L1: Object relationships: Association, Aggregation, Composition.

L2: Strong lifecycle dependency (Composition) vs weak reference (Aggregation).

L3: Reference pointers in garbage collection tracing roots.

L4: Cascade delete database overhead in composition hierarchies.

L5: Modeling aggregate boundaries in microservice databases.

COMMON MISTAKE

Drawing them same in UML. Composition is a filled diamond; Aggregation is an empty diamond.

INTERVIEWER TRAP

Don't confuse the two. If children can exist without the parent, it is Aggregation. If not, it is Composition.

TOP FOLLOW-UPS

- Contrast aggregation vs composition lifecycles.
- What is association vs dependency?

MEMORY TRICK

Aggregation = Cars in a parking lot (lot closes, cars drive away).
Composition = Engine inside a car (crashing the car destroys the engine).

SOLID PRINCIPLES

Yield: ★★★★★

11 – SOLID: S & O Principles

Single Responsibility and Open–Closed design paradigms.

CORE DEFINITIONS

Single Responsibility Principle (SRP): A class should have only one reason to change. Reduces coupling and code drift.

Open–Closed Principle (OCP): Software entities should be open for extension but closed for modification. Avoids breaking existing tests.

OCP Violation:

```
if (shape == "Circle") drawCircle();  
else if (shape == "Square") drawSquare();
```

Fix:

```
Create abstract class Shape, override  
draw()  
in child classes.
```

SRP VIOLATION: GOD CLASSES

A class violates SRP if it has multiple reasons to change. For example, a `UserManager` class that handles user DB actions, sends verification emails, AND logs error files is a God Class that should be split.

WHERE USED IN INDUSTRY

E-commerce checkouts use OCP by defining payment strategy interfaces, allowing new payment methods (Stripe, PayPal) to be added without changing the core checkout orchestrator.

INTERVIEW DEPTH

- L1: Single Responsibility, Open–Closed, Liskov Substitution rules.
- L2: Interface inheritance vs behavioral subtype contract checking.
- L3: Abstracting runtime behaviors using polymorphism to fulfill OCP.
- L4: Complexity overhead of creating multiple classes for SRP.
- L5: Writing regression-free updates in large enterprise frameworks.

CORE INTERVIEW QUESTION

"Explain the Open–Closed Principle (OCP). How do you extend a class's behavior without modifying its source code?"

KEYWORDS TO DROP

POLYMORPHIC ABSTR.

SWITCH CASE SMELL

CODE REGRESSIONS

INHERITANCE EXTENSION

MODEL ANSWER

"OCP states we should extend behavior using polymorphism. Instead of modifying an existing class with conditional checks, we define an abstract interface. New features are added by implementing this interface in new classes, avoiding regressions in existing code."

DEEP DIVE FOLLOW-UP

→ "What is a smell of SRP violation?"

A 'God Class' containing thousands of lines of code handling unrelated tasks (such as managing database connections, processing business logic, AND logging errors).

REAL INTERVIEW SCENARIO

Interviewer: "How do you violate OCP?"

Candidate: "By using complex switch–case or if–else trees inside a core class to handle different behaviors. Adding a behavior requires rewriting the class, risking regression bugs."

COMMON MISTAKE

Violating LSP by making a Square class inherit from Rectangle (setting height changes width unexpectedly).

TOP FOLLOW-UPS

- What is a class cohesion metric?
- Give a real-world SRP violation case.

INTERVIEWER TRAP

Don't apply SRP to the point of creating thousands of tiny classes with single functions; balance modularity with design readability.

MEMORY TRICK

SRP = One tool, one job. OCP = Adding new USB attachments instead of opening the computer to solder components.

SOLID PRINCIPLES

Yield: ★★★★★

12 – SOLID: L, I, & D Principles

Liskov Substitution, Interface Segregation, and Dependency Inversion.

PILLAR DEFINITIONS

Liskov Substitution (LSP): Subtypes must be substitutable for their base types without breaking application logic.

Interface Segregation (ISP): Clients shouldn't be forced to depend on methods they don't use (use small, specific interfaces).

Dependency Inversion (DIP): Depend on abstractions, not concrete implementations.

Dependency Inversion Flow:

[High-Level Module] → [Interface Contract] ← [Low-Level Class]
(Both depend on abstraction, decoupling concrete implementations)

DEPENDENCY INJECTION (DI)

A pattern implementing DIP by passing dependent objects into a class constructor or setter, decoupling object creation from class execution logic.

WHERE USED IN INDUSTRY

Dependency injection containers (like Spring IoC or NestJS) implement DIP to inject mock repository interfaces during testing, decoupling code from database instances.

INTERVIEW DEPTH

- L1:** Interface Segregation and Dependency Inversion rules.
- L2:** Decoupled client interfaces vs fat multi-purpose interfaces.
- L3:** Dependency injection containers and runtime reference wiring.
- L4:** Performance overhead of framework-level bean scanning.
- L5:** Designing pluggable plugins for third-party developer platforms.

CORE INTERVIEW QUESTION

"Explain the Classic Square-Rectangle LSP violation. Why does it violate Liskov Substitution?"

KEYWORDS TO DROP

SUBTYPE INVARIANT

UNEXPECTED SIDE-EFFECT

CONTRACT VIOLATION

ABSTRACT SEPARATION

MODEL ANSWER

"If a Square inherits from Rectangle, setting the width changes the height to maintain square constraints. If a client expects a Rectangle and sets width/height independently, they get unexpected behavior. This violates LSP because the subtype breaks the parent class's invariants."

DEEP DIVE FOLLOW-UP

→ "How does Dependency Injection (DI) relate to DIP?"

DIP is the design goal (depending on abstractions). DI is the mechanism to achieve it (injecting concrete dependencies at runtime via constructor parameters).

SOLID INTEGRATION DEPTH

Level 1 (SRP): Split classes based on actors and business responsibilities.

Level 2 (OCP): Drive interfaces to allow extension without editing core files.

Level 3 (LSP): Ensure subtypes fulfill parent contracts completely.

Level 4 (DIP): Use Dependency Injection to inject mock implementations.

⚠ COMMON MISTAKE

Confusing DIP with Dependency Injection (DI is just a design pattern implementing DIP).

🔄 TOP FOLLOW-UPS

- Explain how DIP differs from DI.
- How does LSP relate to inheritance?

🔴 INTERVIEWER TRAP

Don't confuse LSP with simple overriding compile checks. LSP is a behavioral contract; if subclass overrides break parent assumptions, LSP is violated.

💡 MEMORY TRICK

LSP = Duck test (if it looks like a duck but needs batteries, it is not a true subtype).

LANGUAGE SPECIFICS

Yield: ★★★★★

13 – Access Modifiers

Regulating class visibility boundaries.

VISIBILITY LEVELS

Access modifiers define visibility scopes for classes, variables, and methods.

Private: Accessible only within the declaring class.

Default (Package-Private): Accessible within the same folder/package. No keyword.

Protected: Accessible within the same package + subclasses in different packages.

Public: Accessible globally.

Modifier	Class	Package	Subclass	Global
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

WHY ACCESS CONTROL MATTERS

Restricting visibility reduces coupling. By exposing only public API methods and hiding implementation details under private modifiers, classes can be updated safely without breaking client code.

WHERE USED IN INDUSTRY

SDK library developers use package-private (default) visibility to restrict internal utility classes from being accessed by external API clients, maintaining API safety.

INTERVIEW DEPTH

L1: Private, Default, Protected, Public visibility scopes.

L2: Package-private internal safety vs public API visibility.

L3: JVM verification checks of access control during class loading.

L4: Reflection API bypassing access modifiers (using `setAccessible`).

L5: Encapsulating critical system variables from unauthorized extensions.

CORE INTERVIEW QUESTION

"What is the difference between Protected and Default (Package-Private) access modifiers in Java?"

KEYWORDS TO DROP

SUBCLASS ACCESS

PACKAGE BOUND

FOLDER BOUNDARY

ACCESS HIERARCHY

MODEL ANSWER

"Default access restricts visibility entirely to the declaring package. Protected access extends visibility, allowing subclasses in external packages to access the member variables, which is useful for framework design."

DEEP DIVE FOLLOW-UP

→ "Why can't outer classes be private?"

An outer class marked private would be unusable by any code, rendering it obsolete. Inner nested classes, however, can be marked private.

ACCESS MODIFIER LIMITS

Private: Strictly scoped to class; hidden from nested entities.

Default: Package private; allows fast communication within folders.

Protected: Safe inheritance door; lets subclass hooks extend base models.

Public: Global access; acts as the API face. Keep this as small as possible.

COMMON MISTAKE

Saying protected variables are visible to subclasses only. In Java, they are also visible to any class in the same package.

TOP FOLLOW-UPS

- Explain protected visibility within the same package.
- Can outer classes be private?

INTERVIEWER TRAP

Don't reduce access permissions when overriding a method (e.g. a protected parent method cannot be overridden as private in a child class).

MEMORY TRICK

Private = Self. Default = Family house. Protected = Family house + distant grandchildren. Public = Public street.

METHOD OPERATIONS

Yield: ★★★★★

14 – Static vs Dynamic Binding

Resolving method calls at compile time vs runtime.

CALL RESOLUTION STAGES

Static Binding: Method call is resolved at compile time. Used for static, private, final, or overloaded methods. Faster execution.

Dynamic Binding: Resolved at runtime based on the actual object's type on the heap. Used for overridden methods.

Binding Resolution Timelines:

- **Static (Early):** Source Code → [Compiler Linker] → Target Address
- **Dynamic (Late):** Execution Run → [VTABLE/JVM Lookup] → Target Address

COMPILER OPTIMIZATION (DEVIRTUALIZATION)

Modern Just-In-Time (JIT) compilers analyze inheritance paths. If they prove a virtual method has only one implementation, they bypass the VTABLE check entirely (inlining it) for optimal CPU execution speed.

WHERE USED IN INDUSTRY

Java compiler uses static binding for overloaded helper methods (e.g. Math.max) for speed, and dynamic binding for overriding to enable runtime strategy patterns.

INTERVIEW DEPTH

- L1: Early compile-time binding vs late runtime dynamic binding.
- L2: Static/private binding paths vs virtual/interface dispatch paths.
- L3: Compiler static checks and constant folding optimizations.
- L4: Inlining virtual methods (devirtualization) in JIT compilers.
- L5: Performance tuning of method calls in high-throughput backends.

CORE INTERVIEW QUESTION

"Why are final, static, and private methods statically bound by the compiler?"

KEYWORDS TO DROP

OVERRIDE PREVENTION

CLASS LEVEL BINDING

PERFORMANCE OPT.

EARLY BINDING

MODEL ANSWER

"Static, private, and final methods cannot be overridden by subclasses. Since there is no potential for runtime changes, the compiler resolves their addresses at compile time, bypassing runtime VTABLE lookups."

DEEP DIVE FOLLOW-UP

→ "Which type of binding does overloading use?"

Static binding. The compiler resolves overloaded method calls at compile time based on the argument signatures in the code.

REAL INTERVIEW SCENARIO

Interviewer: "Does Java use static binding for overloaded methods?"

Candidate: "Yes. The compiler evaluates the arguments of the method call at compile-time and maps it to the unique overloaded signature, resolving the call address early."

COMMON MISTAKE

Expecting dynamic binding for private methods. Since private methods aren't inherited, they are bound statically.

TOP FOLLOW-UPS

- Explain why static methods use static binding.
- What is early binding?

INTERVIEWER TRAP

Don't assume dynamic binding is slow. Modern compiler optimizations and JIT compilation make the overhead of dynamic lookup negligible in most apps.

MEMORY TRICK

Static = resolved at the factory (compile). Dynamic = resolved on the road (execution).

MEMORY & MANAGEMENT

Yield: ★★★★★

15 – Garbage Collection & Memory

Automatic heap cleanup vs explicit memory deallocation.

RECLAIMING MEMORY

Garbage Collection (Java/Go): A background daemon thread scans heap memory to identify and delete unreachable objects. Simplifies coding but adds pause times.

Explicit Management (C++): Developers must manually call `delete` to free memory allocated with `new`, or use smart pointers (unique_ptr, shared_ptr).

```
[GC Roots (Active Stack Threads / Statics)]
      ↓ references
[Object A] → [Object B] (Reachable: Safe)
              [Object C] ↔ [Object D]
              (Unreachable: GC Sweep)
```

GENERATIONAL HYPOTHESIS IN JVM

Most allocated objects die young. Modern JVM garbage collectors divide heap memory into three zones: **Eden** (for new objects), **Survivor** spaces, and **Tenured** (for long-lived objects), optimizing scan sweeps.

WHERE USED IN INDUSTRY

Low-latency trading systems tune GC flags (like ZGC or G1GC) to minimize garbage collection stop-the-world pauses, avoiding financial transaction delays.

INTERVIEW DEPTH

- L1:** Reclaiming unused object memory automatically or manually.
- L2:** Reference counting loops vs tracing garbage collection roots.
- L3:** Generational hypothesis: dividing young Eden from tenured space.
- L4:** Stop-The-World latency vs memory compacting performance.
- L5:** Off-heap memory storage for low-latency database caching.

CORE INTERVIEW QUESTION

"How does a Garbage Collector know when an object is eligible for deletion? Explain GC roots."

KEYWORDS TO DROP

REACHABILITY ANALYSIS

REFERENCE COUNTING

GC ROOT NODE

MARK AND SWEEP

MODEL ANSWER

"Garbage collectors trace reachability starting from 'GC Roots' (like thread stack frames or static variables). If an object cannot be reached through any reference path, it is marked unreachable and scheduled for cleanup, resolving circular reference bugs."

DEEP DIVE FOLLOW-UP

→ "What is a memory leak in a garbage-collected language?"

When unused objects remain referenced by active objects (like growing static collections), preventing the garbage collector from reclaiming them.

GC ALGORITHMS DEPTH

- Level 1 (Ref Count):** Count active links; fails on circular reference loops.
- Level 2 (Tracing):** Scan reachable nodes starting from active GC roots.
- Level 3 (Generational):** Divide objects by age into Eden, Survivor, and Tenured spaces.
- Level 4 (JIT/Escape):** Allocate objects directly on the stack via escape analysis.

COMMON MISTAKE

Believing GC guarantees no memory leaks. Forgotten references to unused objects in static collections prevent GC collection.

TOP FOLLOW-UPS

- Explain mark-and-sweep vs reference counting.
- How do memory leaks happen in Java?

INTERVIEWER TRAP

Don't say `System.gc()` forces immediate garbage collection in Java. It only suggests cleanup to the JVM; the engine decides when to run.

MEMORY TRICK

Garbage Collection is a maid service cleaning up after you. Explicit Management is cleaning up your own dishes after every meal.

MEMORY & MANAGEMENT

Yield: ★★★★★

16 – Shallow vs Deep Copy

Copying references vs duplicating nested heap memory.

DUPLICATION MECHANICS

Shallow Copy: Copies the top-level object fields. Nested objects are not duplicated; the copy shares references with the original.

Deep Copy: Recursively duplicates all nested objects, creating completely independent object structures in memory.

Shallow Copy Pointer Sharing

Original.address → [RAM Address Block]
Copy.address → [Shared Block] (Modifications affect both!)

THE JAVA CLONEABLE TRAP

The standard `Object.clone()` is a shallow copy. If a class implements `Cloneable`, it must override `clone()` and manually duplicate nested mutable references to be a true deep copy.

WHERE USED IN INDUSTRY

Redux in React and event sourcing systems require deep copying or immutable updates (using object spread or libraries like Immer) to prevent side-effects in application state.

INTERVIEW DEPTH

- L1: Copying reference addresses vs duplicating memory values.
- L2: Shallow cloning reference arrays vs deep cloning instance trees.
- L3: Object serialization/deserialization copies vs copy factories.
- L4: Memory and CPU overhead of deep clone recursion checks.
- L5: Implementing immutable state managers in reactive applications.

CORE INTERVIEW QUESTION

"Explain the risk of a Shallow Copy when an object contains references to mutable data."

KEYWORDS TO DROP

REFERENCE SHARING

SIDE-EFFECT MODIFICATION

CLONE CONTRACT

MEMORY ISOLATION

MODEL ANSWER

"A shallow copy duplicates only member pointers, not the target data. If the copy modifies a shared mutable object (e.g. updating a list element), it directly alters the original object's state, causing hard-to-debug side-effects."

DEEP DIVE FOLLOW-UP

→ "How is a deep copy implemented in Java?"

By implementing `Cloneable` and overriding `clone()` to manually clone all nested mutable objects, or by using serialization libraries.

REAL INTERVIEW SCENARIO

Interviewer: "How do you perform a deep copy in Java easily?"

Candidate: "We can implement a copy constructor that manually duplicates nested references, or serialize the object to JSON/bytes and deserialize it back into a new object."

COMMON MISTAKE

Modifying a shallow copy, unaware it alters the original object's shared lists/objects.

TOP FOLLOW-UPS

- Contrast shallow cloning vs deep cloning.
- How to implement deep cloning in Java?

INTERVIEWER TRAP

Don't assume `clone()` in Java performs a deep copy by default. The default `Object.clone()` method is strictly a shallow copy.

MEMORY TRICK

Shallow Copy = Sharing a house key. Deep Copy = Building a duplicate house with its own keys.

LANGUAGE SPECIFICS

Yield: ★★★★★

17 – Copy Const. & Assignment

Initializing objects vs copying state into existing objects in C++.

DUPLICATION MECHANICS

Copy Constructor: Initializes a new object as a copy of an existing one (e.g. `Box b2 = b1;`).

Copy Assignment Operator: Copies state from one object into an already initialized object (e.g. `b2 = b1;`). Requires self-assignment checks.

C++ Assignment vs Initialization:

```
Box b1;  
Box b2 = b1; // Copy Constructor (Initialization)  
Box b3;  
b3 = b1; // Copy Assignment Operator (Assigned)
```

RULE OF FIVE (MODERN C++11)

If a class manages system resources, it requires defining five operations: Destructor, Copy Constructor, Copy Assignment, Move Constructor, and Move Assignment to prevent leaks and maximize pointer speed.

WHERE USED IN INDUSTRY

Game engines (like Unity or Unreal Engine) copy complex game objects (e.g. cloning a prefab enemy) using copy constructors to avoid reloading texture assets from disk.

INTERVIEW DEPTH

- L1:** Initializing a new class instance from an existing instance.
- L2:** Deep copy constructors vs compiler-generated default copy.
- L3:** C++ Rule of Three (Destructor, Copy Constructor, Copy Assignment).
- L4:** Prevent compiler recursion by passing copy parameter by reference.
- L5:** Safe resource acquisition and release (RAII) execution.

CORE INTERVIEW QUESTION

"Why does a Copy Constructor accept its parameter by reference rather than by value?"

KEYWORDS TO DROP

INFINITE RECURSION

PASS BY REFERENCE

OBJECT PARAMETER

STACK OVERFLOW

MODEL ANSWER

"If a copy constructor accepted parameters by value, passing an argument would require creating a temporary copy of the object. Creating that copy would call the copy constructor again, leading to infinite recursion and stack overflow."

DEEP DIVE FOLLOW-UP

→ "What is the Rule of Three in C++?"

If a class requires a custom Destructor, Copy Constructor, or Copy Assignment Operator, it almost certainly needs all three to manage resources safely.

RULE OF THREE/FIVE/ZERO

Rule of Three: Custom destructor, copy constructor, and copy assignment are required together.

Rule of Five: Adds move constructor and move assignment for performance.

Rule of Zero: Use smart pointers and standard library templates to avoid manual lifecycle code.

COMMON MISTAKE

Passing by value in copy constructor definitions. Passing by value triggers a copy constructor call, leading to infinite compile-time loops.

TOP FOLLOW-UPS

- Explain the Rule of Three in C++.
- Why must copy constructor pass by reference?

INTERVIEWER TRAP

Don't forget to check for self-assignment (e.g. `if (this == &rhs) return \*this;`) in your assignment operator to prevent deleting your own resources.

MEMORY TRICK

Copy Constructor = Birth by cloning. Assignment = Copying content onto a blank canvas.

OOP Last-Minute Revision Sheet

Core Pillars, Binding Protocols, and SOLID Architectures

⚡ THE 4 PILLARS QUICK-VIEW

Inheritance	Code reuse via hierarchical IS-A relationships.
Polymorphism	Same signature, multiple behaviors (VTABLE override).
Abstraction	Design contracts hiding logic (interfaces/classes).
Encapsulation	Binding state with private scopes & public accessors.

🔧 SOLID PRINCIPLES INDEX

- **Single Responsibility:** Class does one clean task.
- **Open-Closed:** Open for extension, closed to source edit.
- **Liskov Substitution:** Subtypes don't break base invariants.
- **Interface Segregation:** Tiny, focused interfaces.
- **Dependency Inversion:** Code directly to interface templates.

💡 TOP 5 INTERVIEW CONCEPTS TO RECALL

1. **VTABLE Resolution:** C++ objects store a `_vptr` pointing to a static VTABLE of method pointers to resolve overridden functions at runtime.
2. **Diamond Problem:** Ambiguity resolved in C++ via virtual base inheritance, and prevented in Java by banning multiple class inheritance.
3. **Aggregation vs Composition:** Aggregation is a weak HAS-A link (independent lifecycles). Composition is strong, parent-owned.
4. **Rule of Three (C++):** Custom destructor, copy constructor, and copy assignment are required together to manage dynamic memory.
5. **Copy constructor reference:** Must pass by reference (e.g. `Box(const Box& other)`) to prevent infinite recursion.

🎯 QUICK SELF-TEST CHECKLIST

☐ Differentiate aggregation and composition ☐ Trace a VTABLE lookup pointer flow ☐ Explain LSP rectangle-square violation



OOP Comparison Cheat Sheet

Quick Reference Contrast Tables for Fresher Placement Interviews

CLASS VS OBJECT

Feature	Class	Object
Nature	Logical template/blueprint	Physical instance of class
Memory	Occupies no memory	Allocated on heap/stack
Existence	Declared once in code	Instantiated multiple times

CONSTRUCTOR VS DESTRUCTOR

Feature	Constructor	Destructor
Purpose	Initializes object state	Reclaims resource/memory
Trigger	Invoked automatically at new	Invoked at scope exit/delete
Syntax	Same name as class, no return	Preceded by ~ (C++ only)

COMPILE-TIME VS RUN-TIME POLYMORPHISM

Feature	Compile-Time	Run-Time
Resolution	At Compile-Time	At Run-Time
Mechanism	Method Overloading	Method Overriding
Speed	Faster (early static binding)	Slower (indirect VTABLE jump)

METHOD HIDING VS OVERRIDING

Feature	Method Hiding	Overriding
Methods	Redefines parent static method	Redefines parent instance method
Binding	Bound statically to class	Bound dynamically to object
Resolution	Resolved at compile-time	Resolved at runtime (VTABLE)

HEAP VS STACK MEMORY

Feature	Stack	Heap
Contents	Local variables & references	Actual dynamic object instance
Access	LIFO structure, fast execution	Random access, slower pointer jump
Lifetime	Method execution (auto pop)	Garbage collected or manual clean

ABSTRACTION VS ENCAPSULATION

Feature	Abstraction	Encapsulation
Focus	WHAT an object does	HOW state is protected
Level	Design contract interfaces	Implementation-level data shield
Tool	Abstract classes, interfaces	Access modifiers (private)

STATIC VS INSTANCE VARIABLE

Feature	Static Variable	Instance Variable
Ownership	Belongs to class (one copy)	Belongs to object instance (copy/obj)
Location	Metaspace / Class Area	Heap memory inside object
Access	ClassName.variable	objectReference.variable

ASSOCIATION / AGGREGATION / COMPOSITION

Type	Bond Strength	Object Lifecycles
Association	Generic HAS-A	Independent lifecycles
Aggregation	Weak HAS-A	Independent (Professor/Office)
Composition	Strong HAS-A	Dependent (House/Rooms)

SHALLOW COPY VS DEEP COPY

Feature	Shallow Copy	Deep Copy
Method	Copies top references only	Recursively duplicates all nodes
Side-effects	Changes affect cloned copy	Complete insulation from original
Speed	Extremely fast	Slower CPU/memory overhead

COPY CONSTRUCTOR VS ASSIGNMENT OPERATOR

Feature	Copy Constructor	Assignment Operator
Usage	Initializes a new object	Copies state into existing object
Syntax	Box b2 = b1; (New declaration)	b2 = b1; (Pre-existing object)
Self-Check	Not needed	Must check self-assignment (this == &other)

INTERFACE VS ABSTRACT CLASS

Feature	Interface	Abstract Class
Inheritance	Multiple interface implementation	Single base class inheritance limit
State	Only implicit static final constants	Mutable fields & instance variables
Methods	Pure contracts (default/static bodies)	Abstract or concrete method blocks



Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 1)

Q: What is polymorphism, and how does runtime polymorphism differ from compile-time polymorphism?

"Polymorphism means 'many forms' and allows one interface to support different actions. Compile-time polymorphism is resolved during compilation using method overloading, where methods in the same class share a name but have different parameter counts or types, allowing the compiler to bind the call statically. Runtime polymorphism is resolved during execution using method overriding, where a subclass redefines a parent method signature. When a parent reference points to a child instance, the execution environment dynamically redirects the call to the overridden subclass method by looking up its address in the object's VTABLE at runtime."

KEYWORDS:

METHOD OVERLOADING

METHOD OVERRIDING

VTABLE LOOKUP

STATIC BINDING

DYNAMIC DISPATCH

 **Follow-Up:** Can static methods be overridden? What is method hiding?

 **Common Mistake:** Believing static polymorphism uses VTABLE lookup. VTABLEs are exclusive to dynamic runtime overriding dispatch.

 **Expected Depth:** Analyze the CPU branch prediction penalties associated with virtual lookup cache misses.

Q: What is the Diamond Problem in multiple inheritance, and how do C++ and Java handle it differently?

"The Diamond Problem occurs when a child class inherits from two parent classes, which both inherit from a single grandparent class. If the grandparent defines a method and both parents override it, the child class inherits duplicate method templates, causing compiler ambiguity about which version to execute. C++ allows multiple inheritance and resolves this using the virtual keyword during parent inheritance, forcing only a single grandparent instance to compile. Java avoids this completely by banning multiple class inheritance, allowing a class to implement multiple Interfaces instead, where method conflicts must be explicitly overridden by the child."

KEYWORDS:

AMBIGUITY RESOLUTION

VIRTUAL INHERITANCE

MULTIPLE INTERFACES

SINGLE INSTANCE

DIAMOND PATH

 **Follow-Up:** What is a default method in Java interfaces? How are conflicts resolved?

 **Common Mistake:** Claiming Java doesn't support multiple inheritance at all. It supports multiple inheritance of interface types.

 **Expected Depth:** Explain how virtual base class pointers are laid out in memory inside C++ objects.

Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 2)

Q: Explain the Liskov Substitution Principle (LSP) and provide a concrete example of its violation.

"Liskov Substitution Principle, which is the 'L' in SOLID, states that objects of a superclass should be replaceable with objects of a subclass without breaking application correctness. A classic violation is modeling a Square as a subclass of a Rectangle. A Rectangle class lets you set width and height independently. However, a Square forces width and height to remain equal. If client code accepts a Rectangle reference and sets width to five and height to ten, it expects an area of fifty. If we substitute a Square, setting the height overrides the width, yielding an area of one hundred, which violates the parent contract."

KEYWORDS: SUBTYPE SUBSTITUTABILITY RECTANGLE-SQUARE STATE INVARIANTS SOLID PRINCIPLE CONTRACT VIOLATION

Follow-Up: How do you refactor the Square-Rectangle design? Why is composition preferred?

Common Mistake: Thinking LSP is violated only if the code raises a compile error. LSP violations are logical behavioral bugs.

Expected Depth: Contrast behavioral subtyping vs structural inheritance and how compiler check limits relate to them.

Q: What is the difference between an abstract class and an interface, and when would you choose one over the other?

"An abstract class is a partial blueprint that can contain state variables, constructors, and concrete method bodies, and is inherited using single inheritance. An interface is a behavioral contract that is implicitly abstract, has no constructor, holds only static final constants, and can be implemented multiple times. I choose an abstract class when subclasses share common code and identity, establishing an 'IS-A' relationship. I choose an interface to define a decoupled capabilities contract for unrelated classes, establishing a 'CAN-DO' relationship, like making a database connector or file reader Serializable."

KEYWORDS: STATE VARIABLES BEHAVIOR CONTRACT SINGLE INHERITANCE DECOUPLED MODULES IS-A VS CAN-DO

Follow-Up: What is a marker interface? Why did Java 8 introduce static interface methods?

Common Mistake: Saying interfaces cannot have method bodies. Java 8+ supports default and static interface methods.

Expected Depth: Compare performance: abstract class direct calls vs interface invokeinterface opcode lookups in the JVM.

OOP Rapid Fire Questions

Fast-Recall Flashcards for Last-Minute Self-Testing

Q: What is an object?

A: A runtime instance of a class occupying physical memory space.

Q: What is a class?

A: A logical blueprint or user-defined type with no memory usage.

Q: Can a constructor be virtual?

A: No, constructors cannot be virtual because virtual calls require a VTABLE which is initialized by the constructor.

Q: What is method hiding?

A: Defining a static method in a subclass with the same signature as one in the parent class.

Q: What is dynamic binding?

A: Resolving method calls at runtime based on the actual object's type.

Q: What is the diamond problem?

A: Ambiguity arising from multiple inheritance when two parent classes define the same method.

Q: What is a final class?

A: A class that cannot be inherited or subclassed by any other class.

Q: What is composition?

A: A strong HAS-A relationship where child lifecycles are owned by the parent.

Q: What is aggregation?

A: A weak HAS-A relationship where children can survive parent destruction.

Q: What are access modifiers?

A: Keywords defining variable/method visibility scopes (private, default, protected, public).

Q: What is covariant return type?

A: Allowing an overriding method to return a subtype of the parent method's return type.

Q: What is a virtual destructor?

A: A destructor ensuring the child class destructor is invoked during base-pointer deletion.

Q: What is behavioral subtyping?

A: Requiring subclasses to fulfill parent behavioral invariants, not just method signature matches.

Q: What is an interface marker?

A: An empty interface with no methods, used to tag a class as having some capability, like Cloneable.

Q: Can static variables be GCed?

A: Yes, but only when their class loader itself is garbage collected.

Q: What is early vs late binding?

A: Early resolves at compile-time (overloads); late resolves at runtime based on dynamic object type.

Q: Virtual call in C++ constructor?

A: Yes, but it resolves statically to the current class version as derived type is not yet built.

Q: What is Fragile Base Class?

A: When parent class modifications unintentionally break inherited subclasses due to tight coupling.

OOP Common Traps & Deflections

Tactical Responses to Deflect Tricky Placement Questions

Trap 1: The Virtual Constructor Myth

Interviewer: "Why can't we declare a class constructor virtual in C++?"

Deflection: Explain that constructors create the object and initialize the virtual table pointer (`_vptr`). A virtual call requires the `_vptr` and object to already exist in memory, making virtual constructors a logical impossibility.

Trap 2: The Java Interface State Trap

Interviewer: "Can Java interfaces have state variables inside their definition?"

Deflection: No. All fields declared in an interface are implicitly public, static, and final. They are compile-time constants scoped at the class/interface level, not mutable instance state variables.

Trap 3: C++ Copy Constructor Parameter Pass

Interviewer: "Why must copy constructor parameters be passed by reference?"

Deflection: If passed by value, the compiler must copy the argument to pass it. Copying that argument invokes the copy constructor again, initiating another copy, leading to infinite compile-time recursion and stack overflow.

Trap 4: Abstract Class Instantiation Check

Interviewer: "Can we directly instantiate an abstract class?"

Deflection: No. Abstract classes are incomplete blueprints and cannot be instantiated with `new`. However, they do contain constructors, which run automatically during subclass initialization to set inherited fields.

Trap 5: Overriding Static Methods Scope

Interviewer: "Can static methods be overridden in Java subclasses?"

Deflection: No. Overriding depends on dynamic binding of object instances. Static methods are bound statically to the class name at compile-time. Redefining a static method in a subclass is Method Hiding, not overriding.

Trap 6: The VTABLE Memory Overhead Trap

Interviewer: "Does adding virtual methods increase the size of class instances?"

Deflection: No. No matter how many virtual methods are added, only a single virtual table pointer (`_vptr`) is added to each object instance (8 bytes on 64-bit). The VTABLE itself is shared globally at the class level in Metaspace.

Trap 7: Diamond default method conflicts in Java

Interviewer: "What happens if a class implements two interfaces defining default methods with identical signatures?"

Deflection: The compiler throws a conflict error. The class must override the conflicting method and explicitly invoke the desired version using super syntax (e.g. `InterfaceName.super.methodName()`).

Trap 8: C++ Non-Virtual Destructor Memory Leak

Interviewer: "What happens if a base class destructor is not declared virtual when deleting via a base pointer?"

Deflection: It triggers undefined behavior; typically, only the base destructor runs. The derived destructor is bypassed, leaking heap-allocated memory and other resource handles owned by the subclass.

Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 3)

Q: Why is composition preferred over inheritance in OOP system design?

"Inheritance creates a tight, compile-time coupling between classes, often leading to a fragile base class problem where modifying the parent class accidentally breaks child classes. Composition, on the other hand, builds loose coupling by wrapping instances of other classes as member variables. This establishes a 'HAS-A' relationship instead of 'IS-A'. This allows us to swap behaviors dynamically at runtime using interfaces or dependency injection, and prevents deep class inheritance hierarchies that become highly complex to maintain as systems grow."

KEYWORDS: TIGHT COUPLING LOOSE COUPLING FRAGILE BASE CLASS HAS-A VS IS-A RUNTIME STRATEGY SWAPPING

Follow-Up: What is the Liskov Substitution Principle connection here? How do you implement dependency injection?

Common Mistake: Claiming inheritance is completely obsolete. Inheritance is still appropriate for true, stable 'IS-A' hierarchies.

Expected Depth: Analyze the impact on JVM memory allocation and JIT compiler inline optimizations.

Q: What is the difference between shallow copy and deep copy, and what happens in production if we confuse them?

"A shallow copy duplicates only the top-level object itself and copies the reference addresses of any nested objects. This means both the original and copied objects share the exact same child instances in memory, and mutating a child nested field in one object will silently modify the other object. A deep copy recursively duplicates every nested object, ensuring the copied object is completely independent. In production, using a shallow copy by mistake in state-management environments like React Redux can cause state mutations, UI rendering bugs, and database sync conflicts due to shared references."

KEYWORDS: REFERENCE ADDRESS COPY RECURSIVE DUPLICATION SHARED REFERENCES IMMUTABLE STATE STATE MUTATION SIDE-EFFECTS

Follow-Up: How do you implement deep copy in Java? What is the cost of serialization-based deep copy?

Common Mistake: Believing `clone()` in Java performs a deep copy by default. It performs a shallow copy unless explicitly overridden.

Expected Depth: Explain how reference loops are resolved during recursive deep copy traversals.



Personal Notes & Scribbles

For practice questions and mock coding rounds, visit grindos.pranavx.in